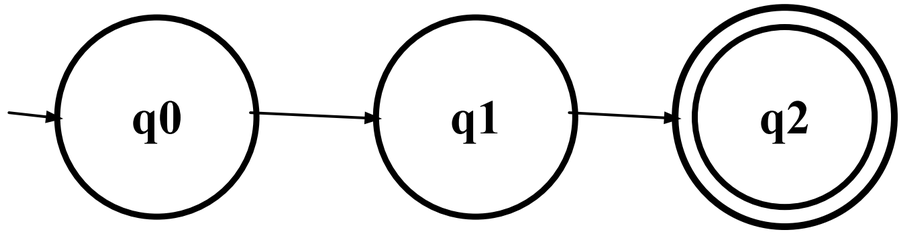
Technical Presentation

# Building a Finite State Automaton for Token Recognition

## DSA0310 - Natural Language Processing

*From regular expressions to validated tokens*




SOURCE

**x1 = 42;**

| x1 | = | 42 | ; |
|---|---|---|---|
| ID | ASSIGN | NUM | SEMI |

**Guided By:**

Dr. Ajitha Rajgopal

**Presented by:**

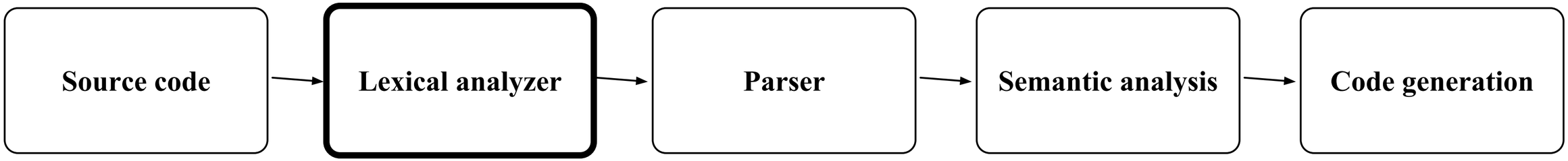
G Chandravardhan Reddy

B Surya Kathyakeya

G Rohit Venkat

# Where the lexer fits

Lexical analysis is the first compiler phase: characters in, tokens out.




**INPUT: A STREAM OF CHARACTERS**

```
i n t   c o u n t   =   1 0 ;
```

No structure yet, just single characters and whitespace.

**OUTPUT: A STREAM OF TOKENS**

| int | count | = | 10 | ; |
|---|---|---|---|---|
| KEYWORD | ID | ASSIGN | INT | SEMI |

Whitespace and comments are discarded along the way.

*Each stage only sees the output of the one before it, so a bad token here becomes a confusing error later.*

# Tokens, lexemes and patterns

Three terms that are easy to mix up, and the whole lexer depends on keeping them apart.

## Token

A category the parser understands.

**IDENTIFIER**

## Lexeme

The exact characters matched in the source.

**count**

## Pattern

The rule describing every valid lexeme of a token.

**letter (letter | digit)***

| Lexeme in source | Token produced | Pattern that accepted it |
|---|---|---|
| **count** | IDENTIFIER | **letter (letter \| digit)*** |
| **10** | INT_LITERAL | **digit+** |
| = | ASSIGN | = |
| **while** | KEYWORD | **reserved word list** |

# Token classes as regular expressions

Every token class gets a regex, and every regex has an equivalent finite automaton.

| Token class | Regular expression | Examples |
|---|---|---|
| **IDENTIFIER** | **[a-zA-Z_][a-zA-Z0-9_]*** | x, _tmp, count2 |
| **INTEGER** | **[0-9]+** | 0, 42, 2026 |
| **FLOAT** | **[0-9]+\.[0-9]+** | 3.14, 0.5 |
| **OPERATOR** | **+ - * / = == <=** | ==, <= |
| **DELIMITER** | **( ) { } ; ,** | ( ; |
| **WHITESPACE** | **[ \t\n]+ (skipped)** | spaces, tabs |

**Each regex becomes one small automaton. The lexer combines them and runs the whole set in a single left-to-right scan.**

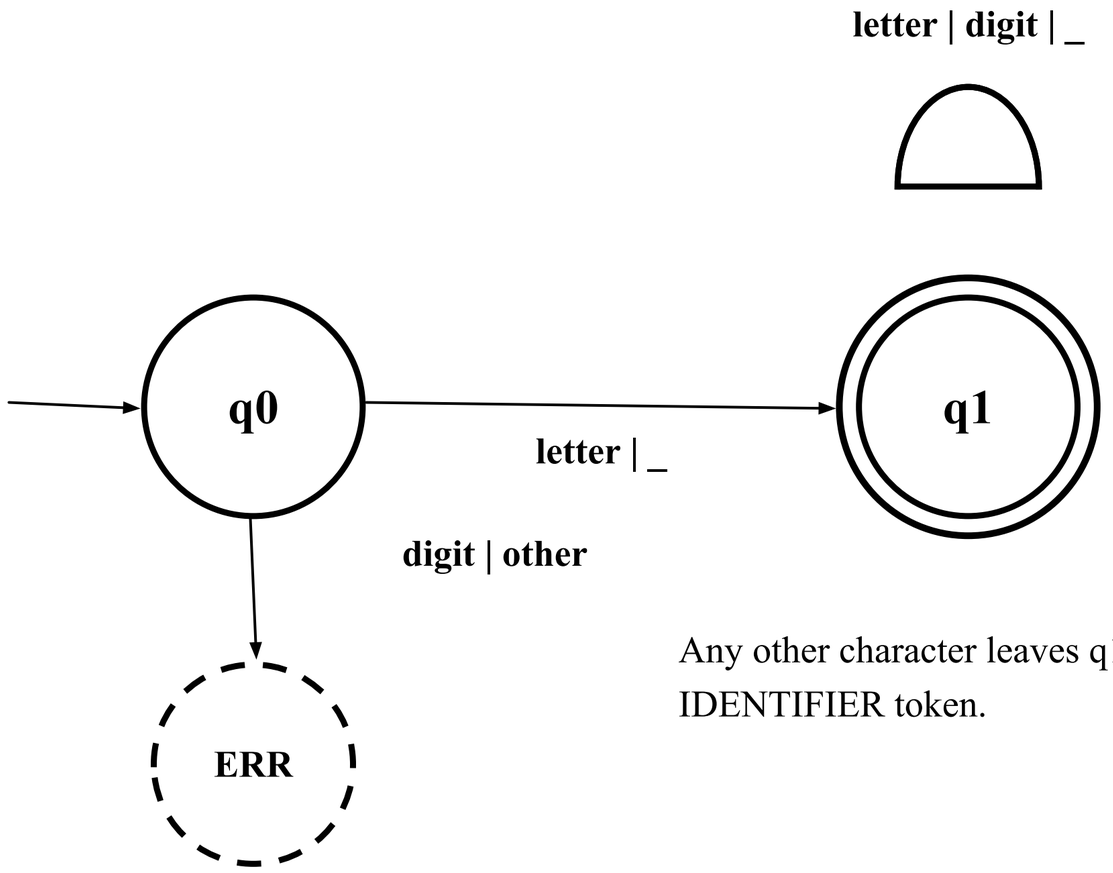
# A DFA is a five-tuple

M = (Q, Σ, δ, q0, F). Each part has a concrete job inside a lexer.

| | FORMAL PART | ROLE IN A LEXER |
|---|---|---|
| Q | **Finite set of states** | How much of a token has been read so far |
| Σ | **Input alphabet** | Characters, grouped into classes: letter, digit, _, other |
| δ | **Transition function $Q \times \Sigma \rightarrow Q$** | One table lookup for each character consumed |
| q0 | **Start state** | Where every new token attempt begins |
| F | **Accepting states** | A complete, valid token has just been recognised |

# DFA for identifiers

Start with a letter or underscore, then allow letters, digits and underscores.




Any other character leaves q1 and emits an IDENTIFIER token.

**TRACE IT**

## count2

q0 → q1 → q1 → q1 → q1 → q1 → q1

**ACCEPTED** **IDENTIFIER**

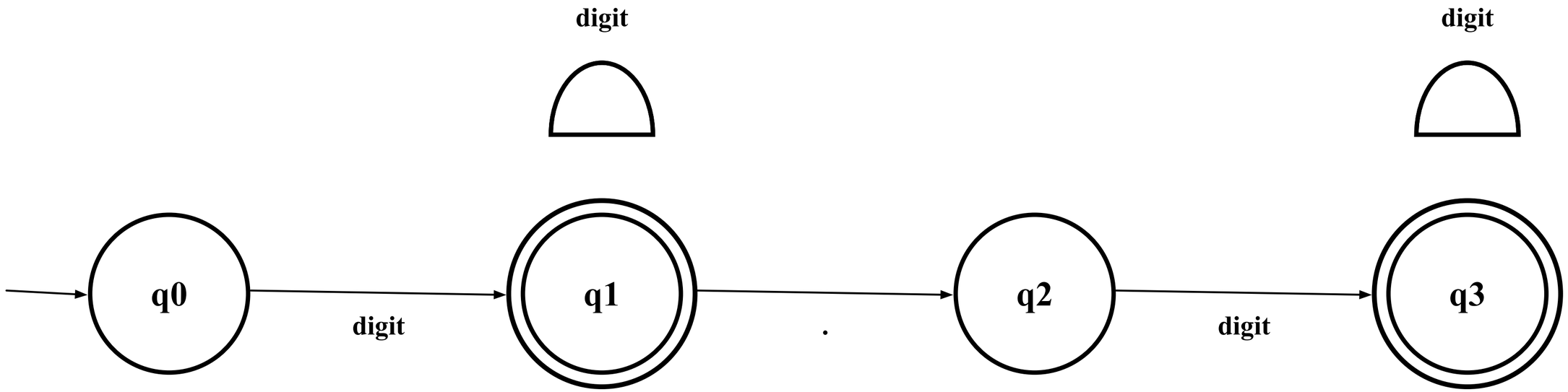
## 2count

q0 –digit→ ERR

**REJECTED** **lexical error**

# DFA for integers and floats

One automaton, two token types. Which accepting state you end in decides the token.




**q1 accepts**

**42** → **INTEGER**

**q3 accepts**

**3.14** → **FLOAT**

**q2 does not accept**

**3.** → **REJECTED**

# Token validation: maximal munch

The lexer always takes the longest valid lexeme, then decides what it means.




### Classify

Read the next character and map it to a class.

### Transition

Follow δ(state, class) to the next state.

### Remember

Whenever the state is accepting, record the position.

### Back up

At a dead end, roll back to the last accepting position and emit the token.

### Keywords vs identifiers

Accept if as an identifier first, then look it up in a reserved-word table. iffy stays an identifier.

### Longest match wins

<= is one operator, not < followed by =. Back up only when the longer path fails.

### Invalid input

A character like @ has no transition. Report line and column, skip it, and keep scanning.

# From automaton to code

Store δ as a table; the scanning loop stays the same for every token class.

| state | letter | digit | _ | other |
|---|---|---|---|---|
| q0 | q1 | ERR | q1 | ERR |
| q1 * | q1 | q1 | q1 | END |

* accepting state.  END means stop and emit the token; ERR means reject.

### Why tables work

One array lookup per character, so scanning is O(n) with no backtracking over the input.

**scanner.py**

```
def next_token(src, i):
    state, last_ok, j = 0, None, i
    while j < len(src):
        nxt = DELTA[state].get(cls(src[j]))
        if nxt is None:
            break
        state, j = nxt, j + 1
        if state in ACCEPT:
            last_ok = (state, j)
    if last_ok is None:
        raise LexError(i)
    return last_ok
```

# Key takeaways

## 1 Tokens are regular

Each token class is a regular expression, so a finite automaton can recognise it.

## 2 States are memory

The current state is all the lexer remembers about the token in progress.

## 3 Accepting means valid

Ending in a state of F is the validation test; anything else is rejected or backed up.

## 4 Longest match wins

Track the last accepting position, then emit the token and resume after it.

**NEXT: NFA to DFA (subset construction) · DFA minimisation · Lex / Flex**

# Thank You!